# OptSem-C: Benchmarking Projection Precision in Public Query-Optimizer Contracts

Haoyi Zhang
Xi'an Jiaotong-Liverpool University
Suzhou, China
hyeliozhang@gmail.com

Huaijin Ran
Nanyang Technological University
Singapore, Singapore
huaijin003@e.ntu.edu.sg

Xunzhu Tang
University of Luxembourg
Luxembourg, Luxembourg
xunzhu.tang@uni.lu

Tegawendé F. Bissyandé
University of Luxembourg
Luxembourg, Luxembourg
tegawende.bissyande@uni.lu

## ABSTRACT

Analytical SQL engines often summarize optimizer behavior with public labels such as `pushdown`, `join`, or `cost based`. Those labels are convenient, but they are not equality predicates: one `pushdown` cell can hide local rewrite, connector delegation, service-side stages, or runtime filtering. OptSem-C treats this as a public-contract denominator problem. It encodes each source-supported optimizer claim as a guarded contract over query features, action kind, optimizer layer, execution placement, decision time, observability, and evidence provenance, then audits which distinctions a comparison vocabulary preserves. The schema, projection portfolio, source manifest, and repair-field universe are fixed before witness counting; robustness checks remain finite-denominator stress tests rather than independent-population generalization. On seven analytical SQL engines, 287 source-linked rules, and 4,216 executable probes, keyword, operator-only, and yes/no vocabularies create 254, 238, and 6 source-supported false equivalences. The 498 headline projection-witness records cover 486 distinct probes; they are denominator errors over the admitted public-contract corpus, not private optimizer bug claims or estimates of proprietary workloads, future engine versions, or operator classes outside the declared schema. Placement repairs keyword collisions, optimizer layer repairs operator-only collisions, and layer plus placement is one interpretable minimum safe basis for the headline records. OptSem-C provides replayable finite counter-witnesses and repair certificates that can be checked before latency, cardinality, or plan-quality comparisons are interpreted.

**PVLDB Reference Format:**
Haoyi Zhang, Huaijin Ran, Xunzhu Tang, and Tegawendé F. Bissyandé. OptSem-C: Benchmarking Projection Precision in Public Query-Optimizer Contracts. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX


Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://doi.org/10.5281/zenodo.21198009. Archive SHA-256: `a0776b2c 41f9cd7985487e7519c47a01b2e30dffaf515c74e286a9b9d8a3ee4b`.

## 1 INTRODUCTION

SQL deliberately hides physical execution choices. That abstraction is useful for applications, but it is too coarse for optimizer comparison. The same query can be accepted by several analytical engines while exposing different public behavior: a local predicate rewrite in one engine, connector-side execution in another, a service-side stage graph in a third, and only a logical explanation in a fourth. These differences determine what a benchmark author, system integrator, or database researcher may conclude about join ordering, pushdown, materialization, adaptive execution, and plan evidence before any deployment-specific performance experiment begins.

The practical precision loss is not cosmetic. A migration checklist may equate local filter rewrite with remote-source delegation; an engine-selection study may compare compile-time join enumeration with runtime repartitioning; a debugging report may treat an estimated plan node and a service profile stage as the same evidence. In audited probe P0009, a keyword view equates BigQuery federated SQL pushdown [16] and query-stage evidence [17] with Trino pushdown [34] and dynamic filtering [33] as `pushdown+explain`. Reference contracts instead separate connector-source filter delegation, stage evidence, aggregate/projection pushdown, and runtime filtering. This is a denominator-validity claim, not an accusation that the cited systems or workloads made that exact mistake: public feature labels and plan surfaces are simply too coarse to serve as equality predicates without a retained-field check. OptSem-C tells such studies which public distinction they preserved and which repair fields make the comparison auditable.

Database research already has precise abstractions for the objects it chooses to study. Relational semantics starts from Codd's model [18]; Selinger-style optimizers separate access-path search from costing [65]; Volcano exposes rule-driven and parallel query processing [36]; the Volcano optimizer-generator account makes extensible search explicit [38]; and Cascades turns that line into a task-driven framework [37]. Chaudhuri surveys optimizer architecture [10]; Ioannidis surveys cost and cardinality assumptions [48]; and Steinbrunn et al. study join-order search heuristics [68]. Public optimizer behavior is usually compared with much coarser vocabularies. A cell labeled "pushdown" may mean predicate movement

inside the engine, partition pruning, aggregation delegation, join delegation, limit delegation, or runtime filtering. A cell labeled "join" may refer to a logical operator, an estimated physical operator, a distributed exchange, or a runtime profile entry. A claim that two engines are "cost based" may hide whether statistics are local, connector-provided, unavailable, or revised during execution. These shortcuts are not harmless summarization: they can create projection-induced contract collisions (a coarse vocabulary declares equality among public contracts whose reference signatures differ).

OptSem-C treats this precision problem as a query-processing systems problem, not as prose cleanup. A *public optimizer contract* is the optimizer behavior an engine exposes through plan interfaces, tuning surfaces, and system descriptions. Documentation can support a contract that is not visible in a particular textual plan, and a plan tag can expose behavior without proving the private cost model. The object is therefore not a hidden implementation trace and not a latency measurement. OptSem-C represents each public statement as a guarded contract rule over query features. The rule records the action, the guard under which it applies, where the action is placed, which optimizer layer owns it, when the decision is made, how the behavior is observable, and whether the public contract requires, permits, forbids, or does not specify the action.

**Research questions.** We ask four finite, auditable questions. RQ1 asks whether public optimizer claims can be encoded as source-linked contracts and executable probes without becoming a latency benchmark. RQ2 asks how often comparison vocabularies that appear in public feature claims collapse distinct source-supported contracts. RQ3 asks which semantic fields eliminate those collisions, and which robustness checks delimit the repair claim. RQ4 asks whether the projection-audit inner loop and SQL-denominator checks are cheap enough to rerun when the public source denominator changes, rather than trusted as a one-off manual inspection.

The central result is a projection-kernel account of these contract collisions. Comparing optimizer contracts through a keyword or operator vocabulary applies a predeclared projection to reference signatures: the full public evidence payload inside the declared schema, not an oracle over private optimizer state. If reference signatures disagree but fall into the same projected class, the projection has declared an equivalence that source-supported public signatures do not support. Contract collisions are therefore denominator errors for the comparison itself; performance or cost measurements should be interpreted only after the compared behavior has been named.

OptSem-C turns that denominator question into a finite comparison relation with executable counter-witnesses, exact repair certificates, and negative controls. The contract schema, source manifest, projection portfolio, motif crosswalk, and repair-field universe are frozen before witness counting; execution checks validate probes but never admit rules or choose repairs. Keyword projection declares 2,284 engine–probe equivalences: 2,030 have matching reference signatures and 254 are separated by reference signatures. Operator-only projection declares 2,268 equivalences, including 238 collisions; the yes/no projection contributes six sparse source-supported records. Across the three vocabularies, the total is 498 projection-witness records over 486 distinct probes, where a record is keyed by projection, engine pair, and probe. Restoring placement repairs keyword collisions; restoring optimizer layer repairs operator-only collisions; retaining layer and placement is one interpretable two-field basis among the enumerated minimum safe sets that repairs all headline records within the admitted public-contract corpus.

The robustness checks are designed to block three tempting overclaims. First, the probe set is a rule-aware coverage benchmark, not an independent population sample, so we do not report statistical generalization beyond the frozen denominator. Second, keyword and operator-only collisions must survive source removal and probe subsampling before they are used as headline evidence; yes/no is reported as source-sensitive because its support is sparse. Third, repair fields are not learned from failed engine-pair transfer: the reported layer-and-placement basis resolves feature-family and engine-family stress splits, while point-learned engine-pair repairs fail on held-out pairs and remain a boundary. A field-only stress test over all 256 retained-field vocabularies leaves only 44 unsafe regions, all covered by concrete counter-witnesses; after excluding action-variant identifiers, no singleton semantic field is safe.

We also introduce OptSemBench-C, a covering benchmark for optimizer-contract behavior. Its denominator is not raw query count. It is coverage of optimizer-relevant feature interactions: source boundary, connector capability, join shape, predicate class, aggregation, statistics need, reuse, distribution, adaptivity, and control surface. The generated probes are executable SQL representatives. The probe space is then cross-checked against DPccp-style join enumeration [59], JOB optimizer evidence [53], later JOB diagnostics [54], exact-cardinality testing [11], progressive optimization [57], mid-query reoptimization [51], adaptive query processing [24], and eddies [3]. We use this audit as a published-motif vocabulary stress test: it asks whether comparison words used around published workloads have concrete semantic requirements in our probe space, not whether the original studies made the same claim or measured the same performance outcome.

**Contributions.** C1 defines a public optimizer contract as a query-processing object separate from SQL result equivalence and runtime performance; Section 3 gives the guarded rule model and the state-join semantics. C2 turns comparison validity into a finite projection audit: reference signatures define the public evidence relation, projected signatures expose counter-witnesses, and repair certificates identify fields that make a vocabulary safe. C3 contributes OptSemBench-C as a denominator design for optimizer-contract comparisons across seven analytical SQL engines; Section 4 reports source-linked admission, executable probes, external-motif coverage, and zero-failure admission checks. C4 reports the projection kernels, anti-overfitting checks, resource profile, and repair certificates: 498 headline projection-witness records, layer and placement as one interpretable minimum safe basis, and cloud replay evidence for the comparison inner loop.

Each central claim is tied to a reproducible record rather than to prose alone. The model section defines the state table and projection obligations; the corpus section exposes source identifiers, line spans, and digest prefixes; the evaluation tables connect SQL execution, motif coverage, collision counts, and repair certificates to regenerated outputs. The case studies then show how those aggregate certificates change comparison judgments.

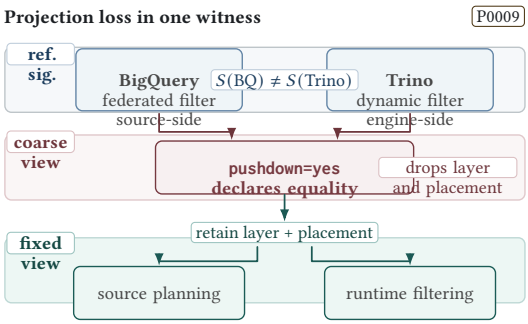


**Figure 1: Projection loss as a finite contract witness. BigQuery federated filtering and Trino dynamic filtering differ under the reference signature, collapse under a coarse `pushdown=yes` denominator, and separate after retaining layer and placement.**

## 2 PRECISION LOSS IN OPTIMIZER COMPARISONS

Figure 1 gives the smallest example. A feature matrix maps several public optimizer contracts into the same label. After the mapping, behaviors with different placement, action kind, and evidence layer become indistinguishable, so a study that treats the label as equality can compare non-equivalent public optimizer behavior. The issue is prior to workload choice: JOB or TPC-H can standardize queries, but they do not say whether two engines expose the same public pushdown, adaptation, or plan-evidence contract.

**Feature-name collision.** A cell such as `pushdown=yes` may denote predicate movement inside an engine, partition pruning, connector-side predicate delegation, aggregation delegation, join delegation, limit pushdown, Top-N delegation, or runtime filtering. These actions affect different operators and have different guards, placements, decision times, and observable plan effects. Runtime and adaptivity distinctions are visible in Tukwila's adaptive integration [49], dynamic query plans [19], and Spark SQL adaptive execution [2]. Rewrite and execution-layer distinctions appear in outerjoin rewrites [35], compiled execution [60], Calcite's adapter-centered optimizer framework [4], Orca's modular optimizer architecture [67], and Dremel-style serving trees [58]. A single feature name can therefore erase exactly the fields a comparison needs.

**Operator disappearance.** A missing native plan node is not a reliable semantics. It may indicate source delegation, logical-plan observability, rewrite, stage-level aggregation, or runtime-only visibility. OptSem-C represents source-delegated execution as a positive contract action rather than as absence. This matters for modern analytical systems whose public surfaces combine Snowflake-style cloud-service stages [22], Redshift's managed warehouse architecture [42], embedded DuckDB execution [62], vectorized MonetDB/X100 execution [6], distributed SQL services such as F1 [66], and dataflow-style execution such as Spark RDDs [73].

**Architecture ranking.** A local engine should not be ranked below a distributed engine merely because broadcast, shuffle, or remote-source delegation is not applicable. Conversely, a federated or cloud engine should not be credited with local optimizer behavior when the public contract exposes delegation or service-side stages. OptSem-C separates non-applicability, documented absence, documented optional behavior, and unspecified evidence.

**Observable contract versus hidden implementation.** The object of study is not what an engine could do internally. It is what the public contract supports as a comparison claim. This distinction is analogous to the separation between SQL equivalence and optimizer-contract equivalence. Cosette proves SQL equivalences by reducing query semantics to logic [15]; semiring-based decision procedures give another equivalence foundation [14]; and QED expands practical SQL equivalence checking [70]. Chandra and Merlin characterize conjunctive-query containment [9]; Aho et al. characterize equivalence among relational expressions [1]. OptSem-C asks a different question: whether the public optimizer behavior being compared exposes the same evidence layers and comparison obligations.

These precision losses imply a minimum payload for comparing public optimizer behavior: guard, canonical action, modality, placement, decision time, observability, version, and provenance. The rest of the paper formalizes this payload and measures what happens when a comparison vocabulary projects it away.

## 3 CONTRACT FORMALIZATION AND NOTATION

A query probe is a SQL skeleton paired with a finite feature vector and validity constraints. Features describe optimizer-relevant structure: join shape and type, predicate class, aggregation, order/limit behavior, source boundary, connector capability, statistics need, reuse structure, distribution trigger, adaptivity trigger, and optimizer control surface. They are not workload-frequency estimates; they are coordinates on the public optimizer decision surface.

The running P0009 witness is a useful concrete instance before the notation. Under a coarse vocabulary, BigQuery and Trino can both appear as `pushdown+explain`. The contract payload keeps the distinctions that the label drops: BigQuery federated SQL records guarded source-boundary filter delegation and service-stage evidence, while Trino records connector pushdown and runtime dynamic filtering under different placement, layer, decision-time, and observability fields. The model below exists to preserve exactly those public distinctions without asserting a private optimizer trace.

Definition 1 (Public contract rule). *Let $\mathcal{A}$ be the finite domain of canonical optimizer actions. An action records operator class, action kind, optimizer layer, execution placement, decision time, observability, and variant. The state set $\mathbb{S}$ contains* MUST, MAY, MUST_NOT, *and* UNSPEC. *A public contract rule is $r = (\gamma, a, s, v, \epsilon)$, where $\gamma$ is a guard over query features, $a \in \mathcal{A}$, $s \in \mathbb{S}$, $v$ is the version or time window, and $\epsilon$ is evidence provenance. For engine $e$, $C_e$ is the finite rule set, and $C_e(q) = \{r \in C_e \mid \gamma_r(\phi(q)) = \text{true}\}$ is the rule set applicable to probe $q$.*

The state domain has two components that feature matrices usually conflate. Behaviorally, MUST requires an action, MUST_NOT forbids it, and both MAY and UNSPEC leave it possible. Evidentially, MAY is public support for optional behavior, while UNSPEC is lack of public support. Equivalently, a state is interpreted as $(m, u, e)$: required behavior, allowed behavior, and evidence support. This

**Table 1: Conflict-aware state join for contract support states. Conflict entries mark evidence combinations that require corpus repair before admission.**

| ⊔ | U | MAY | MUST | N |
|---|---|---|---|---|
| U | U | MAY | MUST | N |
| MAY | MAY | MAY | MUST | C |
| MUST | MUST | MUST | MUST | C |
| N | N | C | C | N |

is why documented optional pushdown and undocumented implementation possibility are different public contracts.

For each engine-probe pair, OptSem-C constructs a contract map $M_{e,q} : \mathcal{A} \to \mathbb{S}$, defaulting every action to UNSPEC. Applicable rules update the map with a conflict-aware state join $\sqcup$. The join strengthens unspecified actions when public evidence is found, preserves compatible stronger evidence, and reports contradictions instead of averaging them. The evidence component follows database-provenance practice: semiring provenance attaches support to derived facts [40], why/where provenance distinguishes explanation locations [7], provenance surveys separate data from explanation [12], and annotated containment keeps evidence annotations visible in reasoning [39]. Figure 2 summarizes how these objects become an audit: source-backed rules are admitted, reference maps are materialized for probes, projected equality is tested against reference inequality, and repair certificates record the retained fields. Table 1 gives the local algebra used inside each map.

Definition 2 (State-join algebra). *The binary operator $\sqcup$ is the complete $4 \times 4$ partial join shown in Table 1. It is total over non-conflicting pairs and returns* CONFLICT *for pairs that simultaneously require and forbid, or support and forbid, the same canonical action. Let $x \preceq y$ denote that $x \sqcup y = y$ for non-conflicting states. Then* UNSPEC *is the least informative state, while* MUST *and* MUST_NOT *are incomparable incompatible commitments.*

Here CONFLICT is a merge-failure signal, not a fifth valid contract state; an admitted corpus must resolve it by revising the conflicting evidence record.

Lemma 1 (Merge invariants). *On non-conflicting states, $\sqcup$ is commutative, associative, idempotent, and monotone with respect to $\preceq$. Therefore a contract map obtained by joining applicable rules is independent of rule order.*

Proof. The properties follow by inspection of the finite table. Idempotence holds because every diagonal entry is the input state. Commutativity holds because the table is symmetric. Associativity and monotonicity are checked over the finite non-conflicting triples; the only excluded triples are exactly those containing incompatible positive and negative commitments. Lifting the pointwise operator to maps preserves the properties for every action independently. □

Let $W \subseteq \mathcal{A}$ be an abstract optimizer-behavior world. A map $M$ denotes the world set $[\![M]\!] = \{W \mid M(a) = \text{MUST} \Rightarrow a \in W,\ M(a) = \text{MUST\_NOT} \Rightarrow a \notin W\}$. The evidential support set is $\text{Supp}(M) = \{a \in \mathcal{A} \mid M(a) \in \{\text{MUST}, \text{MAY}, \text{MUST\_NOT}\}\}$, equivalently the actions where $M(a) \neq$ UNSPEC. Since maps default to UNSPEC, every supported non-default entry arises from an admitted public rule. Behavioral containment and evidential support are intentionally separate: two maps can allow the same worlds while differing on whether an action is publicly documented.

Definition 3 (Reference signatures, kernels, and projections). *The reference evidence-supported signature of $M$ is $S(M) = \{(a, M(a)) \mid a \in \text{Supp}(M)\}$. Reference equivalence is equality of signatures over the full public evidence atom schema. This reference is exact only relative to the admitted public evidence and declared atom fields; it is not a complete private optimizer trace. A projection is a declared map $\pi : \mathcal{A} \times \mathbb{S} \to \mathcal{B}$ from full evidence atoms to an arbitrary comparison vocabulary $\mathcal{B}$, such as keyword, yes/no, or operator-only labels. The projected signature is the finite image $S_\pi(M) = \{\pi(a, M(a)) \mid a \in \text{Supp}(M)\}$. The projection kernel is the equivalence relation $\mathcal{K}_\pi$ where $M_i \mathcal{K}_\pi M_j$ iff $S_\pi(M_i) = S_\pi(M_j)$. A projection-induced contract collision is a pair in $\mathcal{K}_\pi$ whose reference signatures differ.*

For quantitative comparison, exact compatibility is the Jaccard overlap $|S(M_i) \cap S(M_j)|/|S(M_i) \cup S(M_j)|$, and projected compatibility applies the same formula to $S_\pi$. The denominator is evidence-supported: undocumented actions do not count as evidence of either equality or inequality.

Lemma 2 (Public-contract separation). *If $S(M_i) \neq S(M_j)$, then the maps differ in public-contract denotation: either $[\![M_i]\!] \neq [\![M_j]\!]$, or their evidence-supported action states differ for at least one action in $\text{Supp}(M_i) \cup \text{Supp}(M_j)$.*

Proof. If reference signatures differ, some supported action has a state in one map that is absent or different in the other. If the difference changes a MUST or MUST_NOT requirement, the world-set constraints differ. Otherwise the behavioral world set may remain equal, but the public evidence state differs, for example MAY versus UNSPEC. In both cases the denotation differs because an OptSem-C contract is the pair of behavioral worlds and evidence-supported states. □

For finite feature and action domains, constructing $M_{e,q}$ from finite $C_e$ and $q$ is a direct finite evaluation: each guard is a Boolean formula over the probe feature vector, and each applicable rule performs one lookup in the state-join table.

Observation (Projection-kernel collision). *For any projection $\pi$, a projection-induced contract collision is exactly the nontrivial part of the kernel $\mathcal{K}_\pi$: $M_i$ and $M_j$ collide iff $S(M_i) \neq S(M_j)$ and $M_i \mathcal{K}_\pi M_j$. If $\pi$ is non-injective on evidence atoms, there exist contracts with such a collision witness.*

Proof. The first statement unfolds the definitions: $\mathcal{K}_\pi$ is equality after projection, while exact non-equivalence is $S(M_i) \neq S(M_j)$. For the non-injective case, take distinct evidence atoms $(a_1, s_1) \neq (a_2, s_2)$ with $\pi(a_1, s_1) = \pi(a_2, s_2)$. Construct $M_1$ so that $S(M_1) = \{(a_1, s_1)\}$ and $M_2$ so that $S(M_2) = \{(a_2, s_2)\}$, with all other actions UNSPEC. These minimal contracts are well formed because action states are assigned independently. Then $S(M_1) \neq S(M_2)$, but $S_\pi(M_1) = S_\pi(M_2)$ is the same singleton, so $(M_1, M_2)$ is a collision witness. □

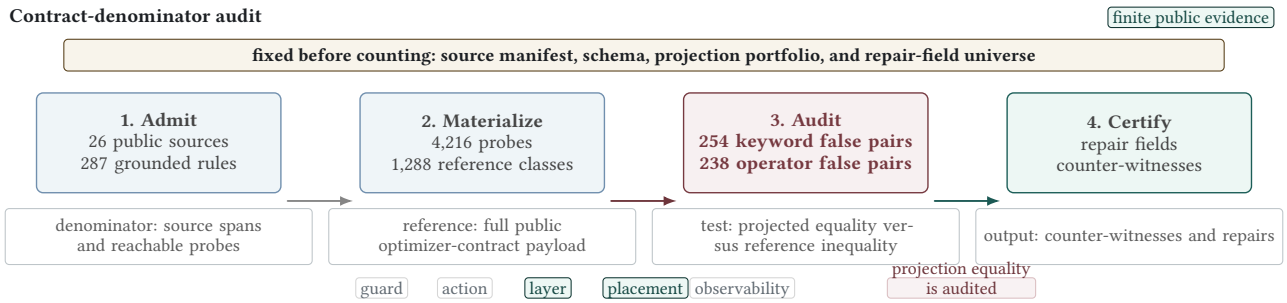


**Figure 2: OptSem-C's contract-comparison dataflow. Inputs are fixed before witness counting; reference signatures define the denominator; projection audits test coarse equality against reference inequality; and each reported repair is tied to regenerated counter-witnesses and source checks.**

Definition 4 (Projection information profile). *For a finite set of observed maps $Y$, let $H(S)$ be the empirical Shannon entropy of reference signatures and $H(S_\pi)$ the entropy of projected signatures. The entropy-retention ratio $\rho_\pi = H(S_\pi)/H(S)$ measures distinguishability retained by the comparison vocabulary. The projected collision count is the number of projected-signature classes containing more than one reference signature class.*

Lemma 3 (Kernel inflation). *If a projection declares $E_\pi$ equivalent pairs and reference comparison declares $E$ over the same pair set, then $E_\pi/E$ is the pairwise inflation factor induced by the kernel. Every collision witness contributes to this inflation, and a zero-collision strengthened projection must have $E_\pi = E$.*

Proof. Reference equivalence implies projected equivalence only when the projection is a function of the full atom payload, so $E_\pi \geq E$. The additional pairs counted by $E_\pi$ are exactly the unequal reference signatures that collapse under $\pi$, which are collision witnesses. If there are none, no additional pairs are counted and the two denominators agree. □

Observation (Evidence refinement). *Adding a non-conflicting evidence-backed rule cannot silently weaken a public contract: it behaviorally refines possible worlds, evidentially refines support, or both.*

The remaining notation is the payoff of the model: it turns an informal vocabulary dispute into a finite certificate check. Let $F$ be the semantic-field universe and let $\pi_R$ append field values in $R \subseteq F$ to the baseline projection. For a finite comparison set $X$, define $X_{\pi_R} = \{(M_i, M_j) \in X \mid S(M_i) \neq S(M_j) \wedge M_i \mathcal{K}_{\pi_R} M_j\}$. A repair $R$ is safe for $\pi$ on $X$ when $X_{\pi_R} = \emptyset$; otherwise any pair in $X_{\pi_R}$ is a counter-witness. Since adding retained fields can split projected classes but cannot merge already separated classes, safe sets are upward closed and unsafe sets are downward closed. For each witness $w$, the separator family $\mathcal{D}_w = \{R \subseteq F \mid M_i \not\mathcal{K}_{\pi_R} M_j\}$ is upward closed, so universal repair is the finite intersection problem $R \in \bigcap_{w \in X_\pi} \mathcal{D}_w$. The reported certificates therefore consist of all repaired witnesses plus counter-witnesses for smaller field sets; exhaustive enumeration over $2^{|F|}$ retained-field subsets is complete for the declared field universe.

Proposition 1 (Minimum repair as hitting set). *For finite semantic fields $F$ and collision witnesses $X_\pi$, deciding whether there is a universal repair of size at most $k$ is NP-complete in $|F| + |X_\pi|$. For OptSem-C's fixed field universe, minimum repairs are exactly enumerable.*

Proof. Membership is finite checking. Hardness reduces Hitting Set: create one witness per set $H_i \subseteq F$ whose two contracts differ exactly on $H_i$; a repair separates that witness iff it intersects $H_i$. □

The semantics makes the comparison payload explicit. A user comparing only operator names chooses a projection that drops placement, decision time, variant, and observability. A feature checklist chooses an even coarser projection that can drop operator class itself. OptSem-C does not forbid those projections; it computes exactly which kernel collisions they induce, which grounded counter-witnesses remain, and which query-processing fields repair the comparison.

## 4 BENCHMARK AND GROUNDED CORPUS

OptSemBench-C is a behavior-contract benchmark: it generates probes that cover optimizer-relevant feature interactions and measures which public contracts those probes activate. It deliberately does not rank engines by latency. This choice follows the benchmark principle that the metric and denominator must match the object of study. CardEst asks whether cardinality estimates improve DBMS plan choice [43]; CardBench stresses learned cardinality estimators in relational settings [13]; Learned Cardinalities models correlated joins [52]; NeuroCard models probabilistic cardinalities [72]; DeepDB learns database distributions [44]; FactorJoin specializes join-cardinality estimation [71]; Neo learns plan choices [56]; and Bao evaluates learned hinting behavior [55]. An optimizer-contract benchmark must instead expose semantic traps in the comparison vocabulary.

The contract schema is fixed before source extraction, witness counting, and repair search. Its fields come from prior optimizer architecture rather than from the measured collisions. Operator/action and layer follow System R [65], Volcano [36], Cascades [37], Calcite [4], and Orca [67]. Placement follows Garlic federation [8], BigDAWG polystores [25], Dremel serving trees [58], and Snowflake

cloud execution [22]. Decision time follows progressive optimization [57], mid-query reoptimization [51], adaptive query processing [24], eddies [3], and Spark adaptive execution [2]. Observability follows PostgreSQL EXPLAIN [41], BigQuery query plans [17], Snowflake EXPLAIN [47], and provenance-style evidence separation [40]. Layer and placement later emerge as repair fields, but enter as predeclared dimensions.

Let $F = F_1 \times \cdots \times F_n$ be the finite feature domain and $\Psi$ the validity constraints. A target interaction is a valid partial assignment over a feature subset. The schema and feature domain are fixed before witness counting and derive from standard query-processing constructs and cited public optimizer surfaces, not from witnesses or motifs. The benchmark targets global pairwise coverage plus selected three-way interactions for source boundary, connector capability, operator class, statistics need, join shape, adaptivity trigger, and distribution trigger. A greedy generator selects renderable full assignments until all renderable target interactions are covered. The generator is rule-aware by design: 99 probes are forced by admitted rule guards so that public contracts are reachable, and the remaining probes are chosen from the feature-interaction universe. This makes OptSemBench-C a coverage benchmark, not an independent held-out generalization set. The generator has two responsibilities. First, it separates benchmark adequacy from any particular system: adequacy is measured against the feature-interaction universe, not against runtime results. Second, it produces probes that are diagnostic rather than workload-frequency estimates. A probe is useful when it activates a contract guard or separates two contract signatures.

Observation (Coverage). *If every valid target interaction has at least one renderable full assignment, the generator terminates with a probe set covering every renderable target interaction.*

Proof. The target universe is finite. Each selected assignment covers at least one uncovered interaction. Thus the number of uncovered interactions decreases monotonically until it reaches zero. □

The main empirical corpus contains only 287 source-linked grounded rules. Each rule records a public evidence span, source URL, line range, digest, and source class. Claims without grounded evidence, expressible guards, or public observability are outside the finite denominator rather than negative evidence. Thus the paper claims coverage of the declared public-contract denominator, not a census of private optimizer rules. Figure 3 gives the denominator view before any collision measurement: sources, rules, probes, and motif requirements are fixed, and the zero-failure checks verify support, consistency, SQL execution, and motif coverage. The corpus covers plan observability, pushdown, join search, distribution, materialization, runtime adaptivity, statistics, and optimizer control surfaces. Every grounded rule is triggered by at least one generated probe, and contract merge produces no conflicts.

The source set is fixed before witness counting. We admit an engine only when public sources provide: a comparable plan or

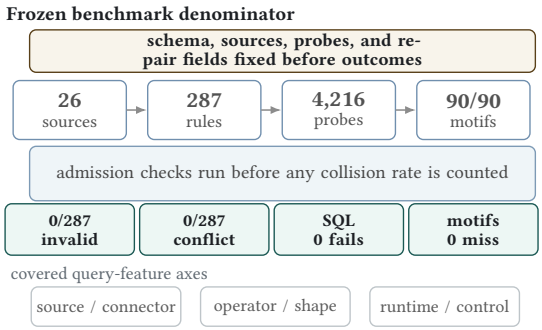


**Figure 3: Corpus, benchmark, and execution denominators. Sources, rules, probes, and motif requirements are fixed before collision measurement; zero-failure checks validate support, consistency, SQL execution, and motif coverage.**

contract surface, enough detail to extract an action and applicability guard, and stable analytical-SQL behavior rather than a private vendor note. PostgreSQL exposes planning and EXPLAIN surfaces [41]; Trino documents cost-based optimization [32], connector pushdown [34], and dynamic filtering [33]; BigQuery publishes query-stage plan explanations [17] and federated SQL pushdown limits [16]; Spark documents adaptive execution [28]; Snowflake documents logical explanation [47]; DuckDB documents CTE materialization controls [30]; and ClickHouse documents join algorithms and optimization guidance [46]. Closed commercial optimizers and framework documents, including Calcite [27] and DataFusion EXPLAIN [29], are schema context rather than denominator engines. The sources are not performance measurements. They provide the public contracts that the framework formalizes. Proprietary workload logs, undocumented cost-model traces, future engine versions, and operator families outside the declared feature domain are excluded rather than silently counted.

The source-linked denominator spans seven engines: PostgreSQL contributes 80 admitted rules, Trino 61, BigQuery 37, Spark SQL 36, ClickHouse and Snowflake 26 each, and DuckDB 21; the largest single source supports 46 rules. This design separates admission, probing, and execution validation. Public sources admit rules into the contract denominator; generated probes test guard reachability and SQL executability; DuckDB and PostgreSQL runs check two open execution surfaces for the generated SQL. A DuckDB source span can admit a DuckDB public contract, but a later DuckDB execution is recorded only as SQL execution and visible-plan sanity evidence. It is not a vote that the source was true, it does not admit rules for any engine, and it is never used to choose a repair field. Conversely, engines without local execution still participate in collision measurements only through source-linked public contracts, source leave-one-out checks, and side-balanced witness support.

The grounded corpus is intentionally conservative. A rule is admitted only when the evidence span supports an optimizer action, a guard or applicability condition, and an observability or placement interpretation. Vague performance advice is excluded from the main corpus. This policy reduces corpus size but strengthens the central claim: the evaluation is about contracts that are publicly supported, not about inferred implementation behavior.

**Table 2: Source-to-rule traceability for representative contracts. Each row binds a rule state/action to a public source span, digest prefix, and paraphrase used by the replay manifest.**

| Contract rule | | | Auditable evidence | | |
|---|---|---|---|---|---|
| Engine | State | Action | Span/digest | | Source-supported paraphrase |
| PostgreSQL [41] | MAY | Plan/observe/local | PG-EXPLAIN-001 bfb47e66 L46–L56 | | EXPLAIN exposes estimated startup and total cost, output rows, row width, and planner-cost units. |
| BigQuery [17] | MAY | Exchange/adapt/cloud | BIGQUERY-PLAN-001 cc7b206a L990–L992 | | The query plan may be modified during execution by adding repartition stages. |
| Snowflake [47] | MAY | Plan/observe/cloud | SNOWFLAKE-EXPLAIN-001 93a61359 L150–L154 | | EXPLAIN compiles a statement and returns a logical execution plan rather than executing it. |
| ClickHouse [46] | MAY | Join/adapt/local | CLICKHOUSE-JOINING-001 a1f757e2 L313 | | With automatic join-algorithm selection, execution can switch from hash join to partial merge join. |
| DuckDB [30] | MAY | Materialize/inline/local | DUCK-WITH-001 L482–L487 | 2a60c8ab | CTE inlining is guarded by single use, absence of volatility, no grouped aggregation, and inlining hints. |
| Spark SQL [28] | MAY | Plan/adapt/distributed | SPARK-PERF-001 L129–L133 | 7e5e2b00 | Adaptive query execution uses runtime statistics and can re-optimize a query while it is running. |
| Trino [34] | MUST_NOT | Agg./pushdown/source | TRINO-PUSH-001 L168–L176 | cc177219 | Aggregation pushdown does not support aggregate calls containing expressions. |

**Contract extraction and validation checks.** Each admitted rule records the engine, version scope, source identifier, digest, action fields, guard, and observability surface. Human coding maps public evidence spans into the fixed schema; automated checks validate schema consistency, source linkage, guard reachability, trigger coverage, merge conflicts, SQL executability, and table-source consistency. This is a source-line admission protocol, not formal verification or recoding agreement: every accepted row is inspectable, and excluded rows are outside the declared denominator. Table 2 gives representative spans; the replay materials carry the complete rule-to-source manifest, source digests, claim-evidence graph, and regenerated table manifests. The field schema and projection portfolio are replay inputs fixed before collision counting. Layer and placement are members of this predeclared field universe; exhaustive enumeration exposes every minimum safe set, and regenerated kernel counts report layer plus placement as an interpretable minimum safe basis rather than a hidden field pair chosen from one witness family. Documentation admits the public claim; EXPLAIN, profile, or stage surfaces bind exposed plan evidence when available; SQL execution validates the probe denominator rather than private optimizer truth. A counted witness requires two source-supported reference signatures, projection equality under the declared vocabulary, and separation by the reported repair field.

The denominator dashboard in Table 3 checks that this conservatism does not create unreachable contracts or syntactic probes. A finite-disjunction guard semantics evaluates each admitted rule against the generated probe denominator. All 287 grounded rules have at least one triggering probe; 252 depend on query features rather than global engine labels; and no guard uses a feature dimension or value outside the benchmark domain.

Coverage is reported over two denominators. The first is benchmark coverage: every targeted renderable valid feature interaction is covered by at least one generated probe. The second is contract triggering: every grounded rule is applicable to at least one probe. Together these denominators make the empirical scope explicit. A rule that cannot be triggered by any probe would indicate a benchmark gap; a feature interaction that cannot be rendered is excluded from the coverage denominator and reported as a modeling constraint.

The benchmark also exposes executable SQL shapes. Each generated probe has a normalized SQL skeleton and shape checks for joins, filters, grouping, ordering, limits, and reuse structure. The full 4,216-probe corpus is accompanied by a compact 48-probe motif-cover prefix. The real-engine motif execution set uses the deterministic union of this motif cover and residual feature-axis maxima; it is not a hand-picked success set, and it is not a replacement for the full evaluation.

A common failure mode in generated benchmarks is to produce feature-complete but non-executable query text. OptSemBench-C therefore validates every SQL skeleton on a deterministic relational catalog with local, remote, and cross-source namespaces. This validation is not a performance baseline and is not exhaustive contract-trigger proof. Its role is stricter and more basic: every generated probe must be parseable, plannable, executable, and shape-consistent with its feature vector. Table 3 reports the validation result: all 4,216 generated probes obtain a plan and execute, with zero SQL-shape failures. Repeating the full corpus across three deterministic catalog populations yields 12,648 successful probe-catalog executions, and DuckDB/PostgreSQL checks add 8,432 full-corpus engine–probe executions plus 142 motif-representative executions with zero execution failures.

Contract validation is layered. Guard reachability says that each public rule has a generated probe satisfying its admitted guard; SQL execution says that the denominator is not syntactic fiction. Visible-plan sanity checks on DuckDB and PostgreSQL expose expected plan tokens at mean rates 0.705 and 0.796 over the full corpus, and 0.756 and 0.819 on motif representatives. These checks do not reverse-engineer private optimizers, prove every documented behavior fires on every cost-model instance, or cross-validate a source by running the same engine. A counted witness still requires source-supported reference signatures, guard reachability, projection equality, and repair separation; runtime plan satisfaction is a separate SQL-denominator validation layer.

The same fixed probes support two budgeted evaluation orders. The cover order is the generator's feature-interaction order. The diagnostic order greedily maximizes newly triggered grounded rules without adding any new probe; it is a rule-coverage schedule over admitted contracts, not an unseen workload sample. Table 4

**Table 3: Benchmark denominator and validation checks across corpus, SQL, real-engine SQL execution, and motif layers. Bold marks complete coverage or zero-failure checks; execution rows validate query construction, not contract truth.**

| Layer | Check | Value | Scope | Interpretation |
|---|---|---|---|---|
| Corpus | grounded rules | **287** | 26 sources | finite public-contract relation |
| Corpus | probe-supported rules | **287/287** | median support 102 | every admitted rule is triggerable |
| Corpus | non-global guards | 252 | 3 containments | query-conditional, audited overlaps |
| Guard schema | invalid dimensions | **0** | fixed feature domain | no out-of-domain guard values |
| SQL corpus | generated probes | **4,216** | digest `df107f018e37` | full executable denominator |
| SQL corpus | motif-cover prefix | 48 | human inspection | compact replay slice, not cherry-picking |
| SQL execution | planned/executed probes | **4,216/4,216** | 91 distinct plans | every generated probe plans and runs |
| SQL execution | shape/exec failures | **0/0** | 18,448 rows | no SQL-shape or runtime failures |
| Real engines | full DuckDB/PostgreSQL | **8,432/8,432** | tags .705/.796 | SQL validation, not source truth |
| Real engines | motif DuckDB/PostgreSQL | **142/142** | tags .756/.819 | budgeted open-engine sanity |
| Published motifs | covered requirements | **90/90** | 12 families | every declared motif has executable probe support |

**Table 4: Probe budgets for triggering grounded contract rules; random columns summarize subset baselines. Bold marks complete trigger coverage.**

| Budget | Cover | Diagnostic | Random mean | Random 5–95 |
|---|---|---|---|---|
| 50 | .672 | .997 | .707 | [.627,.767] |
| 100 | **1.000** | **1.000** | .811 | [.770,.850] |
| 250 | **1.000** | **1.000** | .890 | [.864,.913] |
| 500 | **1.000** | **1.000** | .920 | [.899,.941] |
| 1000 | **1.000** | **1.000** | .951 | [.934,.969] |
| 2000 | **1.000** | **1.000** | .973 | [.958,.986] |
| 4216 | **1.000** | **1.000** | **1.000** | **[1.000,1.000]** |

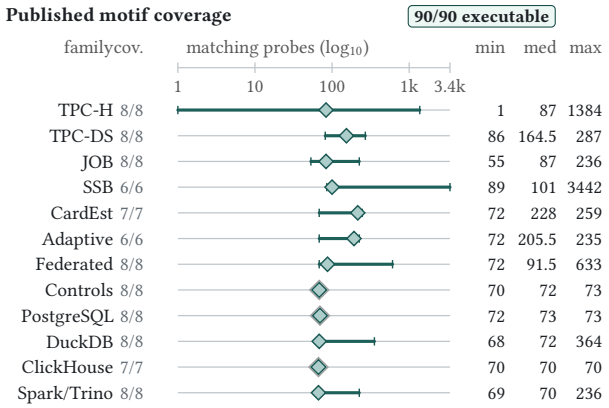


**Figure 4: Published optimizer-motif denominator. Each requirement has an executable representative; teal ranges show minimum–maximum matching probes on a log-scale axis, diamonds mark medians, and ringed diamonds mark collapsed intervals.**

reports both. The cover order triggers all grounded rules within 100 probes; the diagnostic order triggers 286 of 287 rules within 50 probes and all rules within 100. Random subsets of 100 probes trigger 81% of rules on average and 85% at the 95th percentile. Thus OptSemBench-C is not merely large enough to cover the surface; it also contains compact diagnostic prefixes that expose almost all grounded contracts.

A benchmark for public optimizer contracts must connect to existing database workloads without becoming another latency benchmark. We therefore add a crosswalk from published optimizer and benchmark motifs to OptSemBench-C feature requirements, covering JOB join-order sensitivity [53], POLAR-style join-order structure [50], TPC-H [21], TPC-DS [20], SSB [61], LDBC [26], Wisconsin [5], AS3AP [69], ClickBench [45], and LakeBench [23]. Figure 4 reports the full executable denominator: 90 declared requirements across 12 motif families, all covered. Five representative anchors illustrate the span: JOB join-order search (8/8 requirements), exact-cardinality testing (9/9), CardEst-style plan-quality sensitivity (8/8), adaptive replanning (9/9), and cross-source joinability (7/7). Table 5 gives the 12-family difficulty distribution. The selected probes are feature-interaction representatives, not replacements for JOB or TPC-H queries; the crosswalk validates optimizer-decision coverage rather than workload runtime behavior. It remains a frozen audit over optimizer-decision surfaces, not copied benchmark SQL or a latency claim; motifs outside the declared feature domain would be denominator misses. Because motifs are checked after the schema is fixed and before repair fields are used, this is a non-circular denominator check: a counted collision still requires two

**Table 5: Matching-probe difficulty across published optimizer motifs. Underlining marks the sparsest and broadest motif coverage extremes; motif-family sources are cited in the crosswalk paragraph.**

| Suite | Motifs | Min | Median | Max |
|---|---|---|---|---|
| TPC-H | 8 | 1 | 87.0 | 1384 |
| TPC-DS | 8 | 86 | 164.5 | 287 |
| JOB | 8 | 55 | 87.0 | 236 |
| SSB | 6 | 89 | 101.0 | 3442 |
| CardEst | 7 | 72 | 228.0 | 259 |
| Adaptive | 6 | 72 | 205.5 | 235 |
| Federated | 8 | 72 | 91.5 | 633 |
| Controls | 8 | 70 | 72.0 | 73 |
| PostgreSQL | 8 | 72 | 73.0 | 73 |
| DuckDB | 8 | 68 | 72.0 | 364 |
| ClickHouse | 7 | 70 | 70.0 | 70 |
| Spark/Trino | 8 | 69 | 70.0 | 236 |

source-supported signatures, projection equality, side-balanced support, and repair separation. We claim contract-denominator adequacy and published-motif coverage, not third-party workload-performance reproduction.

The motif-difficulty view prevents a misleading denominator. Some motifs are broad and match many probes, such as star-schema group-by shapes. Others are sparse, such as a TPC-H conjunctive-filter motif that has one matching rendered probe under the current validity constraints. Reporting both coverage and matching-probe counts makes benchmark adequacy inspectable: coverage says the motif is present, while difficulty says how concentrated or redundant that coverage is. The executable motif suite adds a second denominator check: for each published motif, the system selects one satisfying probe and validates the generated SQL on the deterministic catalog, yielding 90 validated motif representatives and 383 result rows. Thus the motif denominator is not a list of names; it is a set of optimizer-decision requirements with concrete SQL representatives. Engine-plan validation is a separate experiment because the motif suite itself is a denominator check rather than a claim that published benchmark queries and generated probes induce identical plans.

## 5 EVALUATION

The evaluation follows the objection path a comparison paper would face. Do coarse public vocabularies create denominator errors? Are counts grounded by witnesses rather than names? Do the witnesses survive source, probe, feature, and engine stress without becoming population claims? Are projection-audit stages cheap enough to rerun when the denominator changes? The schema, projections, source manifest, and repair-field universe are frozen before witness counting; execution validates SQL probes but never admits rules, edits projections, or chooses repair fields. The table column "false" is contract-level: a declared vocabulary collapses unequal public optimizer-contract signatures before any latency, plan-quality, or cost claim is interpreted.

The comparison surfaces are executable projection vocabularies and controls, not external DBMS-testing algorithms. Across the 26 public source surfaces, keyword, yes/no, and operator projections mirror feature labels, enablement controls, and operator-family labels; none exposes the complete reference contract payload. Reference comparison is the negative control, one-field projections are diagnostic ablations, and strengthened projections restore fields to localize missing information. All projections are fixed before witness counting and evaluated over the same source-linked maps. The question is how many witnesses are lost if equality is decided at that vocabulary, not whether every cited study made that exact projection. Feature labels come from Trino and Presto documentation [31, 34]; workload and plan-quality controls follow TPC-H/TPC-DS, JOB diagnostics, and exact-cardinality testing [11, 20, 21, 53, 54].

**Coarse comparison induces repairable collisions.** The main empirical result is a finite counterexample to common optimizer-comparison vocabularies. Table 6 keeps public vocabularies, controls, stress rows, and repairs on the same scale: keyword declares 2,284 equivalences and collapses 254 unequal reference signatures, while operator-only declares 2,268 and collapses 238. These errors are conditional on the vocabulary declaring equality, the denominator inherited from a feature matrix. Headline collisions have reference-signature Jaccard distance 1.0 and differ by 9.09 and 5.84 evidence atoms on average. The strict reference and strengthened surface controls have zero false pairs; one-field stress rows show that a single optimizer dimension can make the denominator worse rather than safer.

**Table 6: Main projection audit, combining collision rate, witness severity, and repair diagnosis. Mean/Max Δ count differing public atom fields inside collapsed reference signatures. Stars mark headline public vocabularies; bold numeric zeros mark controls or repairs, and underlining marks the largest unsafe stress row.**

| Projection | Eq. | False↓ | Rate↓ | Mean Δ | Max Δ | Diag. |
|---|---|---|---|---|---|---|
| Exact | 2,030 | **0** | **0.0%** | **0.00** | **0** | ctrl |
| Keyword★ | 2,284 | 254 | 11.1% | 9.09 | 17 | placement |
| Yes/No | 2,036 | 6 | 0.3% | 3.00 | 3 | layer |
| Operator-only★ | 2,268 | 238 | 10.5% | 5.84 | 14 | layer |
| Placement-only | 12,732 | 10,702 | 84.1% | 21.29 | 53 | stress |
| State-only | 25,623 | 23,593 | 92.1% | 13.06 | 64 | stress |
| Surface | 2,030 | **0** | **0.0%** | **0.00** | **0** | repair |

**Table 7: Concrete collision audits linking coarse labels to hidden contract pairs. Repair keys are case-local separators; the reported layer+placement basis covers all headline records.**

| Case | Trace | Repair key | Audit |
|---|---|---|---|
| C1 | P0009; 4 sources | operator; placement; decision time | `pushdown+explain` hides BigQuery source delegation and stage evidence versus Trino connector pushdown and dynamic filtering. |
| C2 | C2; 16 rules | observability; placement | Plan evidence hides local estimates versus cloud-service stages and distributed-worker evidence. |
| C3 | C3; 16 rules | layer; decision time | Adaptive execution hides compile-time planning versus runtime repartition, AQE, and conversion. |
| C4 | C4; 16 rules | variant; guard state | CTE materialization hides inline, materialize, reuse, and disable-guard contracts. |

### 5.1 Concrete Collision Audits

Table 7 grounds the first result with four concrete audits from the witness ledger behind Tables 6 and 16. The cases are deliberately small: each starts from a familiar public label, shows which contracts that label hides, and names the fields that separate the witness. C1 is the running P0009 example, where a coarse `pushdown+explain` label hides BigQuery filter delegation [16], BigQuery stage evidence [17], Trino connector pushdown [34], and Trino dynamic filtering [33]. C2–C4 cover estimate observability, runtime adaptation, and CTE materialization; the same finite checker covers all 498 headline witnesses.

Table 8 shows representative rows from the 17 executable projection surfaces: controls, public-label failures, stress tests, repairs, and the finite resolution frontier. Zero rows are controls or repaired vocabularies; nonzero rows are fixed public projections or retained-field stress tests; repair rows rest on smaller-field counter-witnesses rather than added labels.

Finite resampling keeps keyword at roughly 10–12% and operator labels at roughly 9–12%, with nonzero collisions after any single public source is removed. The repair column localizes the missing semantics: placement repairs keyword, layer repairs the operator

**Table 8: Representative projection surfaces from a 17-row executable portfolio. Bold numeric zeros mark safe controls or repairs; underlining marks the worst shown risk row.**

| Baseline | Eq. | False | Rate |
|---|---|---|---|
| Exact | 2,030 | **0** | **0.0%** |
| Keyword | 2,284 | 254 | 11.1% |
| Op/action | 2,036 | 6 | 0.3% |
| Operator | 2,268 | 238 | 10.5% |
| Kind-only | 2,282 | 252 | 11.0% |
| Layer-only | 2,479 | 449 | 18.1% |
| Place-only | 12,732 | 10,702 | 84.1% |
| State-only | 25,623 | 23,593 | 92.1% |
| Surface | 2,030 | **0** | **0.0%** |

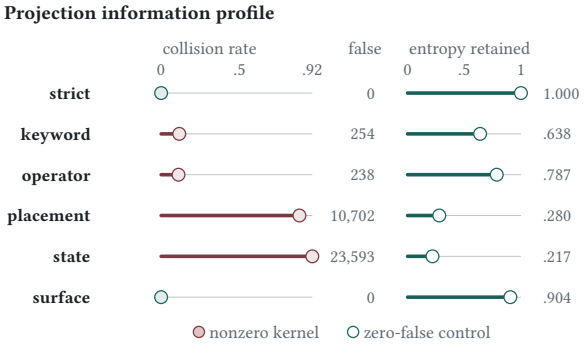


**Figure 5: Projection information profile. Rose bars mark nonzero projection kernels; false-pair counts give witness denominators, while teal controls keep zero false pairs.**

row, and the strengthened semantic surface returns to zero false pairs. The sparse yes/no row remains a narrow witness family.

**Projection kernels are measurable objects.** Figure 5 reports the finite information profile of representative projections. Reference signatures form 1,288 observed classes. Keyword projection compresses them into 148 projected classes, 81 of which contain multiple reference classes, retaining 64% of empirical signature entropy and inflating declared equivalence by 12.5%. Operator-only projection retains more entropy but still leaves 191 colliding projected classes and 238 pairwise collisions. Placement-only and state-only surfaces demonstrate the opposite failure mode: a single field creates very large kernels. The strengthened surface projection still has projected collisions at the signature-class level, but it does not produce any pairwise collision over the engine-probe comparison set. Thus the kernel is finite, populated, and repairable in the observed comparison relation.

**Resolution-lattice stress test.** Table 9 asks which retained-field vocabularies are safe when a comparison may keep any subset of public optimizer fields. The exhaustive test enumerates all 256 subsets over the same 88,536 engine–probe comparisons. This is not a repeated-significance search: every unsafe cell is a concrete counter-witness in a fixed finite relation, so no multiple-testing correction is being used to select a result. The empty vocabulary declares 59,546 unsupported equivalences, and every singleton semantic field remains unsafe: operator leaves 238 collisions, kind 252, layer 449, and placement 10,702. Excluding action-variant identifiers, the

**Table 9: Retained-field lattice for projection repair. Ent. kept is higher-better, Eq. is denominator size, and False is lower-better. All sem. excludes action-variant identifiers, so entropy can remain below 1.0. Bold zeros mark safe vocabularies; underlining marks the largest unsafe collision count.**

| Fields | $\|R\|$ | Classes | Ent. kept↑ | Eq. | False↓ |
|---|---|---|---|---|---|
| None | 0 | 1 | 0.000 | 61,576 | 59,546 |
| Operator | 1 | 387 | 0.787 | 2,268 | 238 |
| Action kind | 1 | 183 | 0.661 | 2,282 | 252 |
| Layer | 1 | 52 | 0.552 | 2,479 | 449 |
| Placement | 1 | 9 | 0.280 | 12,732 | 10,702 |
| Operator+layer | 2 | 591 | 0.850 | 2,030 | **0** |
| Kind+placement | 2 | 214 | 0.680 | 2,030 | **0** |
| Surface | 5 | 820 | 0.904 | 2,030 | **0** |
| All sem. | 7 | 947 | 0.934 | 2,030 | **0** |

**Table 10: Finite-denominator robustness of projection collisions. Down arrows mark lower-better risk columns; bold zeros are controls, not large collision counts. Wilson and Boot. summarize dispersion over declared equivalences and probe clusters, not population inference.**

| Proj. | Type | False↓ | Micro↓ | Wilson | Boot. | Pair↓ | LOO↓ |
|---|---|---|---|---|---|---|---|
| Strict | control | **0** | 0.000 | – | – | 0.000 | – |
| Keyword | label | 254 | 0.111 | [.099,.125] | [.098,.124] | 0.402 | [.017,.144] |
| Yes/No | label | 6 | 0.003 | [.001,.006] | [.001,.006] | 0.003 | [.000,.052] |
| Operator | label | 238 | 0.105 | [.093,.118] | [.094,.121] | 0.290 | [.032,.148] |
| Surface | repaired | **0** | 0.000 | – | – | 0.000 | – |

semantic lattice has 128 subsets, 44 unsafe regions, minimum safe field count two, 14 minimal safe vocabularies, and eight maximal unsafe vocabularies. Ten concrete engine–probe witnesses cover all unsafe retained-field regions, so the monotone frontier certificate is finite rather than anecdotal.

The robustness view in Table 10 checks that the headline result is not an effect of a single denominator. The micro rate conditions on the projection declaring equivalence; Wilson and probe-cluster bootstrap intervals are finite-denominator dispersion summaries over declared equivalences and probe clusters, not population inference; the pair-macro rate averages within engine pairs; and the leave-one-source range removes all rules supported by one public source. Keyword and operator-only collisions remain visible across these views, with leave-one-source conditional-rate ranges $[0.017, 0.144]$ and $[0.032, 0.148]$ respectively.

The evidence side is cross-checked without retraining on headline records: every collision has at least two public source records and side-balanced support. The freeze manifest records the schema, feature domain, source manifest, admitted rules, projection portfolio, motif crosswalk, result-determining code, scripts, and replay entrypoints before witness counting; changing them requires a full replay.

Table 11 is a boundary ledger rather than a victory table. Keyword and operator-only survive every leave-one-source removal, and their 10% probe subsamples remain nonzero in all trials, so the kernel is not produced by one source or probe slice inside the frozen denominator. The yes/no projection is source-sensitive and

**Table 11: Anti-overfitting audit. Boundary labels separate controls, finite-denominator stress, source sensitivity, overlapping folds, and failed learned transfer; no row is an independent population claim.**

| Check | Scope | Evidence | Boundary |
|---|---|---|---|
| Reference ctrl. | strict reference | false=0 | control |
| Source LOO | keyword | nonzero=26/26 | all sources |
| Source LOO | operator-only | nonzero=26/26 | all sources |
| Source LOO | yes/no | nonzero=25/26 | sparse; not all LOO nonzero |
| Probe 10% | keyword/operator 10% | 30/30; 30/30 | within-denom. |
| Feature stress | reported layer+placement | keyword:0; operator-only:0; yes/no:0 | overlapping folds |
| Engine stress | reported layer+placement | keyword:0; operator-only:0; yes/no:0 | within-denom. |
| Learned split | point-learned pair minima | keyword:48; operator-only:120; yes/no:6 | stress exposes non-transfer |

**Table 12: Dispersion of collision witnesses across probes, engine pairs, and feature values. Witnesses are risk counts, not wins; Max=1 for every row, so no probe carries multiple witnesses.**

| Projection | Witnesses | Probes | Pairs | Values | Max/probe |
|---|---|---|---|---|---|
| Keyword | 254 | 254 | 3 | 92 | 1 |
| Operator-only | 238 | 238 | 2 | 75 | 1 |
| Yes/No | 6 | 6 | 1 | 15 | 1 |

stays out of the broad robustness claim. The reported layer-and-placement basis resolves feature-family and engine-family stress, while point-learned engine-pair repairs fail on held-out pairs; that failure keeps the claim finite-denominator rather than learned-transfer.

Because the 287-rule corpus is finite, the remaining checks test whether the kernel is a one-source or one-shape accident. The seven-engine denominator contributes admission evidence, controls, and source-removal stress; the 498 headline false-witness records themselves occupy four observed engine pairs and 486 distinct probes. Table 12 shows that those records touch all 12 feature dimensions and fully cover the value domain for 10 of them.

**Finite-audit replay cost.** A VLDB evaluation should not have to trust an expensive hand audit for the comparison kernel. Table 13 reports cloud replay stages–projection audit, fixed-basis repair, SQL validation, and an 8× deterministic lift–not cold-start setup, archive construction, source extraction, or true workload expansion. The lift repeats the already constructed comparison relation to measure the comparator inner loop; it is not a new corpus, source set, or engine set. Table 14 adds projection-level dispersion: full audits cover 88,536 engine–probe checks per projection above 50,000 checks/s, the 8× lift reaches 708,288 checks per projection above 200,000 checks/s with prefix-scaling $R^2 \geq 0.98$, streaming maintenance has zero drift, and the largest source refresh affects 46 rules, 4,216 maps, and 14.3% of rule–probe rows.

**Repair certificates identify the missing semantics.** Table 6 reports collision counts, severity, and minimum repair diagnosis.

**Table 13: Finite-audit replay cost. Time and RSS are lower-better cloud replay measurements; rows exclude cold-start setup, archive rebuild, paper regeneration, and public-package assembly. The 8× row lifts the same comparison relation.**

| Stage | Input rows | Output rows | Time (s)↓ | RSS (MB)↓ |
|---|---|---|---|---|
| Projection audit | 442,680 | 498 | 1.74 | 275.4 |
| Repair check | 498 | 0 | 0.01 | 275.4 |
| SQL validation | 12,648 | 12,648 | 0.57 | 275.4 |
| 8× audit lift | 2,833,152 | 3,936 | 2.43 | 276.8 |

**Table 14: Finite-comparison replay and dispersion. False and Drift are lower-better; Checks/s is higher-better for the projection inner loop, not end-to-end construction. Bold marks per-column best values.**

| Proj. | False↓ | Probes | Axes | Fam. | Checks/s↑ | Drift↓ |
|---|---|---|---|---|---|---|
| strict | **0** | – | – | 0/6 | 212,618 | **0** |
| keyword | 254 | 254 | 9 | 3/6 | 279,420 | **0** |
| operator-only | 238 | 238 | 8 | 2/6 | **285,041** | **0** |
| surface | **0** | – | – | 0/6 | 236,359 | **0** |

**Table 15: Repair certificate summary. Residuals are lower-better; bold zeros mark complete repair by the reported layer+placement basis. Fold-local chooses repairs within each probe fold.**

| Vocab. | Base↓ | Op./kind↓ | Layer↓ | Place↓ | Fold-local↓ | Basis↓ |
|---|---|---|---|---|---|---|
| Keyword | 254 | 6 | 42 | **0** | 144 | **0** |
| Yes/no | 6 | – | **0** | **0** | **0** | **0** |
| Operator | 238 | 6 | **0** | 32 | 12 | **0** |

Table 15 breaks the evidence into residual paths and probe-fold stability: placement repairs keyword, layer repairs operator-only, and the reported layer-and-placement basis repairs all 498 headline records without fine-grained action-variant labels. Each certificate is a finite proof obligation: counted records match the tested projection, differ under reference contracts, are separated by the repair, and have grounded counter-witnesses against smaller universal repairs.

The residuals show that operator nearly repairs keyword but leaves six collisions, placement eliminates keyword loss, and layer repairs operator-only where placement alone leaves errors. Fold-local repairs can underfit the global denominator; the reported layer-and-placement basis resolves every probe-fold collision. Figure 6 summarizes the frontier: safe retained-field subsets repair every in-scope collision, unsafe subsets retain counter-witnesses, and layer+placement is one interpretable two-field basis among the minimum safe sets.

**Collisions have mechanisms and hotspots.** The repair fields are query-processing boundaries: placement separates local execution, source delegation, distributed workers, and cloud services; layer and decision time separate rewrite, planning, distribution, and runtime adaptation; operator and observability keep estimates, plans, and profiles distinct. Table 16 shows that the highest-attribution fields are also the fields that repair the collisions.

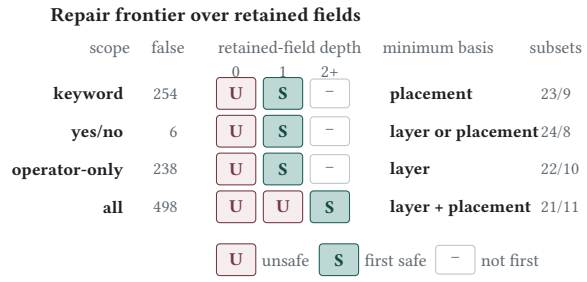


**Figure 6: Repair frontier. U=unsafe, S=first safe repair, and dash=safe non-minimal; *all* uses layer+placement for the 498 headline records.**

**Table 16: Mechanisms and largest engine-pair hotspots. Panel A ranks contract separators; Panel B ranks false-pair concentrations on the same finite witness set.**

| A. mechanism witnesses | |
|---|---|
| Mechanism | W. |
| Action variant | **498** |
| Placement | 466 |
| Optimizer layer | 430 |
| Plan evidence | 366 |
| Decision time | 362 |
| Operator family | 281 |

| B. pair hotspots | |
|---|---|
| Hotspot | False↓ |
| Keyword BQ–CH | **203** |
| Keyword BQ–Trino | 45 |
| Keyword CH–Trino | 6 |
| Yes/No CH–Trino | 6 |
| Operator CH–Trino | 123 |
| Operator Spark–Trino | 115 |

**Table 17: Controlled workload-feature movements over frozen toggle neighborhoods. Groups and pairs give finite support; bold marks the mean-distance sort key.**

| Engine | Feature | Groups | Pairs | Mean↑ | Nonzero↑ |
|---|---|---|---|---|---|
| DuckDB | reuse structure | 289 | 1,380 | **0.83** | 0.85 |
| Trino | statistics need | 260 | 1,382 | 0.74 | 0.85 |
| DuckDB | source boundary | 57 | 61 | 0.59 | 0.62 |
| Trino | distribution trigger | 237 | 1,097 | 0.52 | 0.86 |
| Spark SQL | adaptivity trigger | 235 | 1,075 | 0.44 | 0.86 |
| BigQuery | distribution trigger | 237 | 1,097 | 0.37 | 0.89 |
| Spark SQL | join type | 229 | 1,201 | 0.37 | 0.81 |
| ClickHouse | order/limit | 89 | 487 | 0.35 | 0.51 |

**Optimizer contracts are workload-sensitive.** Table 17 reports controlled feature-toggle movements over CTE reuse, source boundary, statistics need, distribution, and adaptivity. The largest movements are finite neighborhoods over the frozen feature space, so comparability is a relation between engine contracts and query structure.

## 6 DISCUSSION AND LIMITATIONS

**Scope and protocol.** Public optimizer claims should name action, guard, layer, placement, decision time, and evidence surface. Before ranking latency, cardinality error, or plan quality, a study should test that shared contract relation, then retain separating fields, narrow the claim, or report mixed phenomena. JOB, TPC-H, and TPC-DS standardize inputs and metrics, not contract equality; OptSem-C is the pre-performance denominator audit.

**Boundary of the claim.** OptSem-C does not infer private cost models, rank workload performance, or turn a finite public corpus into a population guarantee. Failed engine-pair stress, source sensitivity, and replay cost stay visible; broader corpora, proprietary workloads, future engine versions, and out-of-schema operators require new admission. The resource tables measure replayable comparison stages and SQL-denominator checks, not full setup.

**Implication for benchmarks.** The intended use is a precondition for ordinary performance evaluation. A benchmark paper can run the contract admission step on its public feature matrix, report the unsafe label groups, and either keep the separating fields or state that the row mixes several phenomena. This does not weaken latency or cardinality results; it makes clear which optimizer behavior they describe. For a new engine or source surface, the protocol is append-only: add public records, replay the frozen projections, and accept a new repair only if it still separates the old and new witnesses. The same discipline helps evaluators distinguish a real optimizer change from a changed public label.

**Validity reading.** A false pair is not an execution bug, and a zero row is not an accuracy theorem; both are statements about equality under the admitted public-contract relation. This reading matters as datasets grow: more sources may add witnesses, while stronger public APIs may remove collisions by exposing placement, layer, or evidence fields. We therefore treat every count as replayable evidence over a named denominator. The result is strongest where independent stresses agree–source removal, probe subsampling, engine-family stress, SQL execution, and deterministic scaling–and weakest where the table itself reports transfer failure or source sensitivity. Those weak points stay in the main body so users know when to rerun admission before interpreting performance plots.

## 7 RELATED WORK

Optimizer architectures supply the schema fields. System R, Volcano/Cascades, Calcite/Orca, Garlic/BigDAWG, Snowflake, and Dremel expose access paths, rule search, adapters, placement, and service stages [4, 8, 22, 25, 36, 37, 58, 65, 67]. OptSem-C turns these public dimensions into comparison-contract fields without claiming private optimizer observability.

Benchmarks ask downstream questions: TPC-H/TPC-DS define workloads [20, 21]; JOB, CardEst, and CardBench stress join order, cardinality, and plan quality [13, 43, 53, 54]; Neo and Bao study learned plan choice or hinting [55, 56]. OptSem-C checks whether public labels preserve the same contract semantics before those metrics are interpreted.

SQL equivalence and testing systems such as Cosette, QED, SQLancer, and NoREC target query semantics or DBMS bugs [15, 63, 64, 70]. They do not define equality for optimizer-contract labels such as pushdown, runtime adaptation, or plan evidence.

**Conclusion.** OptSem-C checks whether public optimizer labels provide a valid denominator before latency, cardinality, or plan-quality comparisons. Its replayable ledger records unsafe labels, witnesses, repair fields, and stress checks. It tells authors what fields a comparison preserves and tells evaluators whether a dispute concerns the denominator or runtime evidence.

# REFERENCES


[1] A. V. Aho, Y. Sagiv, and J. D. Ullman. 1979. Equivalences among Relational Expressions. *SIAM J. Comput.* 8, 2 (1979), 218–246. https://doi.org/10.1137/0208017

[2] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *SIGMOD*. 1383–1394. https://doi.org/10.1145/2723372.2742797

[3] R. Avnur and J. M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *SIGMOD*. 261–272. https://doi.org/10.1145/342009.335420

[4] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *SIGMOD*. 221–230. https://doi.org/10.1145/3183713.3190662

[5] D. Bitton, D. J. DeWitt, and C. Turbyfill. 1983. Benchmarking Database Systems: A Systematic Approach. In *VLDB*. 8–19. https://www.vldb.org/conf/1983/P008.PDF

[6] P. A. Boncz, M. Zukowski, and N. Nes. 2005. MonetDB/X100: Hyper-Pipelining Query Execution. In *CIDR*. 225–237. https://www.cidrdb.org/cidr2005/papers/P19.pdf

[7] P. Buneman, S. Khanna, and W.-C. Tan. 2001. Why and Where: A Characterization of Data Provenance. In *ICDT*. 316–330. https://doi.org/10.1007/3-540-44503-X_20

[8] Michael J. Carey, Laura M. Haas, Peter M. Schwarz, Manish Arya, William F. Cody, Ronald Fagin, Myron Flickner, Allen W. Luniewski, Wayne Niblack, Dragutin Petkovic, John Thomas, John H. Williams, and Edward L. Wimmers. 1995. Towards Heterogeneous Multimedia Information Systems: The Garlic Approach. In *RIDE-DOM*. 124–131. https://doi.org/10.1109/RIDE.1995.378736

[9] A. K. Chandra and P. M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. In *STOC*. 77–90. https://doi.org/10.1145/800105.803397

[10] S. Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. In *PODS*. 34–43. https://doi.org/10.1145/275487.275492

[11] S. Chaudhuri, V. R. Narasayya, and R. Ramamurthy. 2009. Exact Cardinality Query Optimization for Optimizer Testing. *PVLDB* 2, 1 (2009), 994–1005. https://doi.org/10.14778/1687627.1687739

[12] J. Cheney, L. Chiticariu, and W.-C. Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474. https://doi.org/10.1561/1900000006

[13] Y. Chronis, Y. Wang, Y. Gan, S. Abu-El-Haija, C. Lin, C. Binnig, and F. Ozcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. arXiv. arXiv:2408.16170 https://arxiv.org/abs/2408.16170 Accessed 2026-06-16.

[14] S. Chu, B. Murphy, J. Roesch, A. Cheung, and D. Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *PVLDB* 11, 11 (2018), 1482–1495. https://doi.org/10.14778/3236187.3236200

[15] S. Chu, C. Wang, K. Weitz, and A. Cheung. 2017. Cosette: An Automated Prover for SQL. In *CIDR*. 1–10. https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf

[16] Google Cloud. 2026. Introduction to Federated Queries. Documentation. https://docs.cloud.google.com/bigquery/docs/federated-queries-intro Accessed 2026-07-04.

[17] Google Cloud. 2026. Query Plan and Timeline. Documentation. https://cloud.google.com/bigquery/docs/query-plan-explanation Accessed 2026-06-16.

[18] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387. https://doi.org/10.1145/362384.362685

[19] R. L. Cole and G. Graefe. 1994. Optimization of Dynamic Query Evaluation Plans. In *SIGMOD*. 150–160. https://doi.org/10.1145/191839.191872

[20] Transaction Processing Performance Council. 2024. TPC Benchmark DS Standard Specification. TPC. https://www.tpc.org/tpcds/ Accessed 2026-06-16.

[21] Transaction Processing Performance Council. 2024. TPC Benchmark H Standard Specification. TPC. https://www.tpc.org/tpch/ Accessed 2026-06-16.

[22] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *SIGMOD*. 215–226. https://doi.org/10.1145/2882903.2903741

[23] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *PVLDB* 17, 8 (2024), 1925–1938. https://doi.org/10.14778/3659437.3659448

[24] A. Deshpande, Z. Ives, and V. Raman. 2007. Adaptive Query Processing. *Foundations and Trends in Databases* 1, 1 (2007), 1–140. https://doi.org/10.1561/1900000001

[25] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. 2015. The BigDAWG Polystore System. *SIGMOD Record* 44, 2 (2015), 11–16. https://doi.org/10.1145/2814710.2814713

[26] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC Social Network Benchmark: Interactive Workload. In *SIGMOD*. 619–630. https://doi.org/10.1145/2723372.2742786

[27] Apache Software Foundation. 2026. Apache Calcite Documentation. Documentation. https://calcite.apache.org/docs/ Accessed 2026-06-16.

[28] Apache Software Foundation. 2026. Performance Tuning: Spark 4.1.2 Documentation. Documentation. https://spark.apache.org/docs/4.1.2/sql-performance-tuning.html Accessed 2026-06-16.

[29] Apache Software Foundation. 2026. Reading Explain Plans. Documentation. https://datafusion.apache.org/user-guide/explain-usage.html Accessed 2026-06-16.

[30] DuckDB Foundation. 2026. DuckDB Documentation: WITH Clause and CTE Materialization. Documentation. https://duckdb.org/docs/stable/sql/query_syntax/with.html Accessed 2026-06-16.

[31] Presto Foundation. 2026. Cost Based Optimizations: Presto 0.298.1 Documentation. Documentation. https://prestodb.github.io/docs/current/optimizer/cost-based-optimizations.html Accessed 2026-06-16.

[32] Trino Software Foundation. 2026. Cost-Based Optimizations: Trino 482 Documentation. Documentation. https://trino.io/docs/482/optimizer/cost-based-optimizations.html Accessed 2026-06-16.

[33] Trino Software Foundation. 2026. Dynamic Filtering: Trino 482 Documentation. Documentation. https://trino.io/docs/current/admin/dynamic-filtering.html Accessed 2026-07-04.

[34] Trino Software Foundation. 2026. Pushdown: Trino 482 Documentation. Documentation. https://trino.io/docs/current/optimizer/pushdown.html Accessed 2026-07-04.

[35] Cesar A. Galindo-Legaria and Arnon Rosenthal. 1997. Outerjoin Simplification and Reordering for Query Optimization. *ACM Transactions on Database Systems* 22, 1 (1997), 43–74. https://doi.org/10.1145/244810.244812

[36] G. Graefe. 1990. Encapsulation of Parallelism in the Volcano Query Processing System. In *SIGMOD*. 102–111. https://doi.org/10.1145/93597.98720

[37] G. Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Engineering Bulletin* 18, 3 (1995), 19–29. https://www.sigmod.org/publications/dblp/db/journals/debu/Graefe95a.html

[38] G. Graefe and W. J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *ICDE*. 209–218. https://doi.org/10.1109/ICDE.1993.344061

[39] T. J. Green. 2011. Containment of Conjunctive Queries on Annotated Relations. *Theory of Computing Systems* 49, 2 (2011), 429–459. https://doi.org/10.1007/s00224-011-9327-6

[40] T. J. Green, G. Karvounarakis, and V. Tannen. 2007. Provenance Semirings. In *PODS*. 31–40. https://doi.org/10.1145/1265530.1265535

[41] PostgreSQL Global Development Group. 2026. PostgreSQL Documentation: 18: 14.1. Using EXPLAIN. Documentation. https://www.postgresql.org/docs/18/using-explain.html Accessed 2026-06-16.

[42] Anurag Gupta, Deepak Agarwal, Derek Tan, Jakub Kulesza, Rahul Pathak, Stefano Stefani, and Vidhya Srinivasan. 2015. Amazon Redshift and the Case for Simpler Data Warehouses. In *SIGMOD*. 1917–1923. https://doi.org/10.1145/2723372.2742795

[43] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *PVLDB* 15, 4 (2021), 752–765. https://doi.org/10.14778/3503585.3503586

[44] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, and C. Binnig. 2020. DeepDB: Learn from Data, Not from Queries! *PVLDB* 13, 7 (2020), 992–1005. https://doi.org/10.14778/3384345.3384349

[45] ClickHouse Inc. 2024. ClickBench: A Benchmark for Analytical DBMS. Online benchmark. https://benchmark.clickhouse.com/ Accessed 2026-06-16.

[46] ClickHouse Inc. 2026. Using JOINs in ClickHouse. Documentation. https://clickhouse.com/docs/guides/joining-tables Accessed 2026-06-16.

[47] Snowflake Inc. 2026. EXPLAIN: Snowflake Documentation. Documentation. https://docs.snowflake.com/en/sql-reference/sql/explain Accessed 2026-06-16.

[48] Y. E. Ioannidis. 1996. Query Optimization. *Comput. Surveys* 28, 1 (1996), 121–123. https://doi.org/10.1145/234313.234367

[49] Z. G. Ives, D. Florescu, M. Friedman, A. Levy, and D. S. Weld. 1999. An Adaptive Query Execution System for Data Integration. *ACM SIGMOD Record* 28, 2 (1999), 299–310. https://doi.org/10.1145/304181.304209

[50] David Justen, Daniel Ritter, Campbell Fraser, Andrew Lamb, Allison Lee, Thomas Bodner, Mhd Yamen Haddad, Steffen Zeuch, Volker Markl, and Matthias Boehm. 2024. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance. *PVLDB* 17, 6 (2024), 1350–1363. https://doi.org/10.14778/3648160.3648175

[51] N. Kabra and D. J. DeWitt. 1998. Efficient Mid-Query Re-Optimization of Sub-Optimal Query Execution Plans. In *SIGMOD*. 106–117. https://doi.org/10.1145/276304.276315

[52] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *CIDR*. 1–11. https://www.cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf

[53] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2015. How Good Are Query Optimizers, Really? *PVLDB* 9, 3 (2015), 204–215. https://doi.org/10.14778/2850583.2850594

[54] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *VLDB Journal* 27, 5 (2018), 643–668. https://doi.org/10.1007/s00778-017-0480-7

[55] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *SIGMOD*. 1275–1288. https://doi.org/10.1145/3448016.3452838

[56] R. Marcus and O. Papaemmanouil. 2019. Neo: A Learned Query Optimizer. *PVLDB* 12, 11 (2019), 1705–1718. https://doi.org/10.14778/3342263.3342644

[57] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdzic. 2004. Robust Query Processing Through Progressive Optimization. In *SIGMOD*. 659–670. https://doi.org/10.1145/1007568.1007642

[58] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2010. Dremel: Interactive Analysis of Web-Scale Datasets. *PVLDB* 3, 1-2 (2010), 330–339. https://doi.org/10.14778/1920841.1920886

[59] G. Moerkotte and T. Neumann. 2006. Analysis of Two Existing and One New Dynamic Programming Algorithm for the Generation of Optimal Bushy Join Trees without Cross Products. In *VLDB*. 930–941. https://dblp.org/rec/conf/vldb/MoerkotteN06.html

[60] T. Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *PVLDB* 4, 9 (2011), 539–550. https://doi.org/10.14778/2002938.2002940

[61] P. O'Neil, E. O'Neil, X. Chen, and S. Revilak. 2009. The Star Schema Benchmark and Augmented Fact Table Indexing. In *TPCTC*. 237–252. https://doi.org/10.1007/978-3-642-10424-4_17

[62] M. Raasveldt and H. Muehleisen. 2019. DuckDB: An Embeddable Analytical Database. In *SIGMOD*. 1981–1984. https://doi.org/10.1145/3299869.3320212

[63] M. Rigger and Z. Su. 2020. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *ESEC/FSE*. 1140–1152. https://doi.org/10.1145/3368089.3409710

[64] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *OSDI*. USENIX Association, 667–682. https://www.usenix.org/conference/osdi20/presentation/rigger

[65] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *SIGMOD*. 23–34. https://doi.org/10.1145/582095.582099

[66] Jeff Shute, Radek Vingralek, Bart Samwel, Ben Handy, Chad Whipkey, Eric Rollins, Mircea Oancea, Kyle Littlefield, David Menestrina, Stephan Ellner, John Cieslewicz, Ian Rae, Traian Stancescu, and Himani Apte. 2013. F1: A Distributed SQL Database That Scales. *PVLDB* 6, 11 (2013), 1068–1079. https://doi.org/10.14778/2536222.2536232

[67] Mohamed A. Soliman, Lyublena Antova, Venkatesh Raghavan, Amr El-Helw, Zhongxian Gu, Entong Shen, George C. Caragea, Carlos Garcia-Alvarado, Foyzur Rahman, Michalis Petropoulos, Florian Waas, Sivaramakrishnan Narayanan, Konstantinos Krikellas, and Rhonda Baldwin. 2014. Orca: A Modular Query Optimizer Architecture for Big Data. In *SIGMOD*. 337–348. https://doi.org/10.1145/2588555.2595637

[68] M. Steinbrunn, G. Moerkotte, and A. Kemper. 1997. Heuristic and Randomized Optimization for the Join Ordering Problem. *VLDB Journal* 6, 3 (1997), 191–208. https://doi.org/10.1007/s007780050040

[69] C. Turbyfill, C. Orji, and D. Bitton. 1993. AS3AP: An ANSI SQL Standard Scaleable and Portable Benchmark for Relational Database Systems. In *The Benchmark Handbook for Database and Transaction Processing Systems*. 95–121. https://www.odbms.org/2014/03/benchmark-handbook-1993/

[70] S. Wang, S. Pan, and A. Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *PVLDB* 17, 11 (2024), 3602–3614. https://doi.org/10.14778/3681954.3682024

[71] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27. https://doi.org/10.1145/3588721

[72] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica. 2019. Deep Unsupervised Cardinality Estimation. *PVLDB* 13, 3 (2019), 279–292. https://doi.org/10.14778/3368289.3368294

[73] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *NSDI*. 15–28. https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia